

# MutoRS Core Source Code Interpretation

## MutoRS Core Source Code Interpretation

1. Overview
2. In-depth analysis of the project directory structure
3. In-depth Analysis of Core Source Code
  - 3.1. Integration of ROS 2 and FastAPI (`muto_controller_node.py`)
  - 3.2. Command Execution and Task Preemption (`command_executor.py`)
  - 3.3. Voice Interaction (`voice_assistant_service.py`)
  - 3.4. Robot Motion Encapsulation (`MutoLargeModelInterface.py`)
4. 从语音到动作的简单示例

## 1. Overview

MutoRS uses a large language model (based on Dify's LLM) as the brain, and ROS 2 as the cerebellum and body, conducting neural transmission through a standardized HTTP interface.

- **Brain:** Runs locally on the Dify Workflow (LLM), responsible for intent recognition, task breakdown, and motion planning.
- **Body:** Runs on the robot's ROS 2 node, responsible for sensor perception (vision, hearing), low-level motion control (hexapod gait), and execution feedback.
- **Nerve:** Communicates full-duplex via HTTP (FastAPI), exchanging instruction sequences and execution status using JSON format.

## 2. In-depth analysis of the project directory structure

```
~\muto-llm-2.0
├─ muto_llm_ws/                                # ROS 2 workspace
│   └─ src/muto_ros2_controller/
│       └─ muto_ros2_controller/
│           └─ muto_controller_node.py         # [Core] Master node, integrating
FastAPI Server
│               └─ command_executor.py         # [Core] Command execution engine,
parsing JSON and scheduling
│               └─ voice_assistant_service.py  # [Core] Voice service, handling
wake-up and STT in a separate thread
│               └─ routes.py                   # [Interface] Defines HTTP routes
such as /execute_commands
│               └─ camera_manager.py           # [Vision] Camera process management,
preventing resource conflicts
│               └─ models.py                   # [Data] Define the Pydantic data
model (Request/Response)
├─ packages/                                   # Standalone Python packages
(hardware abstraction layer)
│   └─ muto_hexapod_lib/                       # Core control library for the
hexapod robot
│       └─ muto_hexapod_lib/
```

```

|   |   |   |   └─ MutoLargemodelInterface.py # [Interface] High-level motion
encapsulation (forward, rotate, squat)
|   |   |   |   └─ core/                        [Driver] Low-level servo driver,
serial communication, kinematics calculation
|   |   |   |   └─ Largemodel/                    # [Function] Navigation, memory,
sensor management module
|   |   |   |   └─ voice_module/                  # Voice interaction module
|   |   |   |   └─ voice_module/
|   |   |   |   └─ voice_interface.py             # [Interface] Unified voice interface
(wake/STT/TTS)
|   |   |   |   └─ core/                          # [Implementation] Alibaba
Cloud/iFlytek SDK Encapsulation
|   |   |   |   └─ config/                        # [Configuration] Voice Parameter
Configuration (voice_config.yaml)
└─ prompt_backup_en/                            # LLM Prompt words (Prompt
Engineering)
|   └─ LLM/
|   |   └─ Decision_Layer_Prompts.md              # Responsible for breaking
down natural language into steps
|   |   └─ Execution_Layer_Prompts.md             # Responsible for
converting steps into Python function calls (JSON)
|   └─ env/                                       # System Environment Description and
Capability Documentation
└─ rebuild_and_launch.sh                        # One-click build and startup script

```

## 3. In-depth Analysis of Core Source Code

### 3.1. Integration of ROS 2 and FastAPI (muto\_controller\_node.py)

This is the system entry point. MutoRS directly starts a `FastAPI` instance in the ROS 2 Node.

#### Code Analysis:

```

class MutoControllerNode(Node):
    def __init__(self):
        super().__init__('muto_controller_node')

        # 1. Initialize the instruction executor, which is the core object
        connecting ROS and the underlying driver.
        self.command_executor = MutoCommandExecutor(logger=self.get_logger(),
        camera_manager=self.camera_manager)

        # 2. Start the FastAPI application
        self.app = FastAPI(title="Muto Robot Controller API")

        # 3. Register the route and inject the command_executor into the route
        handling function.
        self.routes = MutoAPIRoutes(self.app, self.command_executor,
        logger=self.get_logger())

        # 4. Set a timer for a health check
        # Check the status of each component every 300 seconds to ensure long-
        term operational stability.
        self.create_timer(300.0, self.health_check)

```

The advantage of this design is that the Web interface can directly call ROS's publish/subscribe functionality without inter-process communication (IPC), thus reducing latency.

## 3.2. Command Execution and Task Preemption (command\_executor.py)

This is the core module, responsible for handling commands issued by the LLM. We implemented a key mechanism here: **task preemption based on RequestID**, to solve the problem of "how to be interrupted while speaking/moving".

### Core Logic:

1. **Preparation Phase (prepare\_for\_new\_request)**: When a new task arrives (such as user voice wake-up), the old task is immediately and forcibly terminated.

```
def prepare_for_new_request(self, new_request_id: str):
    """
    Prepare for the new request ID: Force interrupt the old task
    """
    # 1. Trigger a global cancel signal
    self.session_manager.request_cancel()

    # 2. Physical Stop Subsystem: Stop voice playback and stop robot servo
    motor movement.
    self._stop_subsystems()

    # 3. Update RequestID
    # Key point: Older asynchronous tasks check the ID before proceeding to
    the next step; if the ID has changed, they will automatically exit.
    self.update_request_id(new_request_id)

    # 4. Clear the cancellation flag to allow new tasks to begin execution.
    self.clear_cancel()
```

2. **Execution Phase (execute\_command)**: Parses the JSON instructions and maps them to Python functions.

```
async def execute_command(self, plan_item: CommandPlan) -> Dict[str, Any]:
    # 1. High-priority cancellation check: Confirm again before execution
    whether it has been cancelled.
    if self.cancel_event.is_set():
        return {'success': False, 'error': 'Execution interrupted'}

    # 2. Parse the instruction string, for example, "forward(steps=2)"
    command_str = plan_item.command.strip()
    func_name = command_str.split('(')[0].strip()

    # 3. Dynamically calling MutoInterface methods
    # For example, func_name='forward', params={'steps': 2}
    if func_name == 'forward':
        # Ultimately, the underlying library packages/muto_hexapod_lib is
        called.
        result = self.muto_interface.forward(**kwargs)
        return result
```

```

elif func_name == 'have_a_look':
    # Visual Task: Take a photo -> Upload -> Analyze
    return self._execute_have_a_look(plan_item.id, user_query)

```

### 3.3. Voice Interaction (voice\_assistant\_service.py)

This service runs in a separate thread.

#### Code Explanation:

```

def _wake_loop(self):
    while self.is_running:
        # 1. Listen for wake words (Snowboy/Porcupine)
        success, _, detected = self.voice.is_wake_detected()

        if success and detected:
            # 2. Core: Generate a new ID and interrupt the ongoing action!
            request_id = str(uuid.uuid4())
            self.command_executor.prepare_for_new_request(request_id)

            # 3. Play the response sound effect "I'm here"
            self._play_wake_ack()

            # 4. Entering the single-round interaction process
            self._single_interaction(request_id)

def _single_interaction(self, request_id):
    # 1. Recording
    self.voice.start_single_segment_recording(...)
    # 2. STT (Speech-to-Text)
    text = self.voice.recognize_audio_file(...)
    # 3. Submit to backend for processing (Dify)
    self._start_backend_processing(text, request_id)

```

### 3.4. Robot Motion Encapsulation (MutoLargemodelInterface.py)

Located in `packages/muto_hexapod_lib`, this file encapsulates complex inverse kinematics solutions into a human-readable high-level API.

#### Code Explanation:

```

class MutoController:
    # Unified Action Interface
    def forward(self, steps: Optional[int] = None, distance_m: Optional[float] =
None, ...):
        """Forward movement"""
        # Calculate steps or distance by calling the underlying robot.move()
function.
        result = self._execute_move_action('forward', steps, distance_m, time_s,
speed_ms)

        # Physical reset: Ensure the robot returns to a standing position after
the action is completed, rather than remaining tilted.
        self.robot.move(0, 0, 0)
        time.sleep(0.05)
        self.robot.stay_put()
        return result

```

## 4. 从语音到动作的简单示例

When the user says "Take two steps forward", the internal data flow of the system is as follows:

### 1. Command Input

- `VoiceAssistantService` detects the wake word and calls `executor.prepare_for_new_request` to stop the current action.
- Records the message and calls STT to obtain the text "Take two steps forward".

### 2. Execution

- `commandExecutor` sends the text to the Dify API.
- **LLM (Execution Layer Prompt)** generates JSON based on the prompt:

For simplicity, in Dify, the actual process involves decision-making + execution + iterative result evaluation.

```

{
  "status": "success",
  "plan": [
    { "id": 1, "command": "robot_speak(text='好的，向前走两步') " },
    { "id": 2, "command": "forward(steps=2)" }
  ]
}

```

### 3. Schedule

- JSON returned to `routes.py` -> `execute_commands`.
- `command_executor` receives the list of commands.

### 4. Action

- **Command 1:** `robot_speak` -> Calls the TTS service to play audio.
- **Command 2:** `forward(steps=2)` -> Calls `MutoController.forward` -> Calculates inverse kinematics -> Drives 18 servos.

### 5. Feedback

- Execution results (success/failure) are logged. If the LLM determines that multiple steps are needed (e.g., "take two steps and then look around"), these results are returned to the LLM as context for further planning.